NORTH BENGAL INTERNATIONAL UNIVERSITY

Lab Report : 5

TOPIC : WRITE A PYTHON PROGRAM TO FIND THE ROOTS OF AN EQUATION OF REGULAR FALSE METHOD WHEN THE NUMBERS ARE INPUT BY THE USER

COURSE CODE : MATH 0541-2212

COURSE TITLE : NUMERICAL ANALYSIS SESSIONAL

Submitted By:

MD. TANVIR RAHMAN PRANTO

0712340012101005

CSE -26

Submitted To:

SAIFUR RAHMAN

LECTURER, CSE

Write a python program to find the roots of an equation of Regula Falsi method when the numbers are input by the user:

Theory:

The Regula Falsi method, also known as the False Position method, is a numerical technique used to find the root of a function. This method is particularly useful for solving equations where analytical solutions are difficult or impossible to obtain. The Regula Falsi method is based on the intermediate value theorem, which states that if a continuous function changes sign over an interval, there is at least one root in that interval.

Key Concepts:

1. Intermediate Value Theorem:

The method relies on the Intermediate Value Theorem, which states that for a continuous function f(x), if f(a) and f(b) have opposite signs, there exists at least one root in the interval [a, b].

2. Initial Interval:

Start with two points, a and b, such that f(a) and f(b) have opposite signs (f(a) * f(b) < 0). This ensures that a root lies between a and b.

3. Linear Interpolation:

The method approximates the root by finding the x-intercept of the line connecting the points (a, f(a)) and (b, f(b)). The formula for the approximation c is:

$$c = b - \frac{f(b) \cdot (a - b)}{f(a) - f(b)}$$

4. Iteration:

- Calculate the new approximation c.
- Check if f(c) is sufficiently close to zero (within a specified tolerance).
- Update the interval [a, b]:
  - If f(c) * f(a) < 0, set b = c.
  - If f(c) * f(b) < 0, set a = c.

5. Convergence:

The process is repeated until the difference between consecutive approximations is less than the tolerance, indicating that the root has been found with the desired accuracy.

Advantages:

- The Regula Falsi method often converges faster than the Bisection method because it uses the actual function values for interpolation.
- It guarantees convergence if the function is continuous and the initial guesses bracket a root.

Limitations:

- The method may converge slowly if the function is not well-behaved or if the initial guesses are not close to the root.

- It requires that the function changes sign over the interval, which might not always be the case.

Algorithm:

Algorithm for the Regula Falsi Method:

1. Start.

2. Define the function `f(x)` that represents the equation x^3 - x - 1.

3. Input values:
- Prompt the user to enter the left endpoint a.
- Prompt the user to enter the right endpoint b.

4. Set the tolerance to 10^-7.

5. Check if the initial interval is valid:
- If f(a) * f(b) >= 0, print "No root in that interval" and stop.

6. Initialize:
- Calculate the initial approximation x.

7. Iteration:
- Repeat until the absolute difference between successive approximations is less than the tolerance:
  - Increment the step counter.
  - Calculate the new approximation x_0.
  - Print the current step and x_0.
  - If f(a) * f(x_0) < 0, update b to x_0.
  - Else if f(b) * f(x_0) < 0, update a to x_0.
  - Else, print the current step and x_0, and break.
  - Calculate the next approximation x.

8. Print the result:
- If the root is found, print the final approximation.

9. End.

Code:

```
# Function representing the equation
def f(x):
    return x**3 - x - 1  # Example equation

# Input values from the user
a = float(input("Enter the left endpoint a: "))
b = float(input("Enter the right endpoint b: "))
tolerance = 10**-7

# Check if the initial interval is valid
if f(a) * f(b) < 0:
    x = (a * f(b) - b * f(a)) / (f(b) - f(a))
    i = 0
    while True:
        i += 1
        x0 = x
        print(f'Step {i}: x = {x0}')

        if f(a) * f(x0) < 0:
            b = x0
        elif f(b) * f(x0) < 0:
            a = x0
        else:
            print(f'Step {i}: x = {x0}')
            break

        x = (a * f(b) - b * f(a)) / (f(b) - f(a))

        if abs(x - x0) < tolerance:
            print(f'Step {i}: x = {x0}')
            break
else:
    print('No root in that interval')
```

Output:

```
OUTPUT    TERMINAL
PS C:\Users\Lenovo> & C:/Users/Lenovo/AppData/Local/Programs/Python/Python313/python.exe c
Enter the left endpoint a: 1
Enter the right endpoint b: 2
Step 1: x = 1.1666666666666667
Step 2: x = 1.2531120331950207
Step 3: x = 1.2934374019186834
Step 4: x = 1.3112810214872344
Step 5: x = 1.3189885035664628
Step 6: x = 1.322282717465796
Step 7: x = 1.3236842938551607
Step 8: x = 1.3242794617319507
Step 9: x = 1.3245319865800917
Step 10: x = 1.3246390933080368
Step 11: x = 1.3246845151666695
Step 12: x = 1.3247037764713758
Step 13: x = 1.3247119440797213
Step 14: x = 1.3247154074520977
Step 15: x = 1.3247168760448744
Step 16: x = 1.324717498779053
Step 17: x = 1.3247177628396751
Step 18: x = 1.324717874810383
Step 18: x = 1.324717874810383
PS C:\Users\Lenovo>
```

Conclusion: In conclusion, this Python code implements the Regula Falsi method to find the root of a given equation. By iterating through linear interpolation, it approximates the root and provides detailed step-by-step outputs. This approach showcases an effective numerical technique for solving equations, emphasizing the method's practicality and convergence properties.